

Algorytmy zaawansowane - projekt Ping-Pong.

Dokumentacja końcowa

Ewelina Preś, Stanisław Boczarski, Dominik Dygas

21 maja 2026

1 Zmiany względem dokumentacji wstępnej

Jedna ze zmian w naszym sposobie rozwiązywania dotyczy reprezentacji macierzy zwycięstw i porażek. Początkowo chcieliśmy zastosować konwencję, że zawodnik przegrywa sam ze sobą, co w macierzy objawiało się zerami na przekątnej macierzy:

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 \end{pmatrix}$$

Rozwiązanie problemu można było wtedy znaleźć przeszukując macierz A oraz obliczoną macierz A^2 , gdyż zachodziło wówczas:

$$\text{Zawodnik } i \text{ ma własność } X, \text{ jeśli } \forall j \neq i : A_{ij} = 1 \vee A_{ij}^2 > 0. \quad (*)$$

Końcowo jednak postanowiliśmy reprezentować macierz zwycięstw i porażek w następujący sposób:

$$A' = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix}$$

czyli stosując konwencję, że zawodnik wygrywa sam ze sobą, co widać po jedynekach na przekątnej macierzy. Poniżej prezentujemy uzasadnienie dokonanej zmiany:

Niech $A' := A + I$.

Wtedy $A'^2 = (A + I)^2 = A^2 + AI + IA + I^2 = A^2 + 2A + I$

W ten sposób do macierzy A^2 modelującej „zwycięstwa pośrednie” dodajemy podwójnie macierz A modelującą zwycięstwa bezpośrednie. Zatem niezerowa wartość elementu $(A'^2)_{ij}$ oznacza, że zawodnik i wygrał pośrednio lub bezpośrednio z zawodnikiem j . Zachodzi wówczas:

Zawodnik i ma własność X , jeśli $\forall j \neq i : (A'^2)_{ij} > 0$.

Stosowanie takiego podejścia pozwala na przeszukanie tylko kwadratu macierzy A' zamiast - jak w poprzednim podejściu - przeszukiwać obie macierze A i A^2 .

Druga zmiana dotyczy formatu plików z danymi wejściowymi. W obecnej wersji linie pliku opisujące macierz zwycięstw składają się z wartości 0 lub 1 oddzielonych od siebie przecinkiem oraz spacją, a cała linia jest objęta nawiasem kwadratowym.

Przykład zmiany:

Plik z danymi, który w poprzedniej wersji miał zawartość

```
5
1 1 1 0 1
0 1 1 1 1
0 0 1 0 1
1 0 1 1 0
0 0 0 1 1
```

teraz wygląda następująco

```
5
[1, 1, 1, 0, 1]
[0, 1, 1, 1, 1]
[0, 0, 1, 0, 1]
[1, 0, 1, 1, 0]
[0, 0, 0, 1, 1]
```

2 Generator plików wejściowych

Jako osobny program powstał kod generujący pliki z danymi wejściowymi o nazwie *Ping-pong-generator.py*. Po jego uruchomieniu użytkownik jest proszony o podanie liczby uczestników turnieju. Należy podać liczbę całkowitą większą od 1. Liczby mniejsze od 2, niecałkowite lub wartości nieliczbowe nie zostaną przyjęte i program będzie ponawiał prośbę, aż do otrzymania poprawnej wartości. Następnie użytkownik jest proszony o wpisanie nazwy pliku, do którego chce zapisać dane. Program automatycznie dodaje do nazwy pliku rozszerzenie *.txt*, jeżeli nie zostało ono podane. Kod generuje losową macierz modelującą przebieg turnieju, zgodną ze zmianami opisanymi w sekcji 1. Plik o podanej nazwie, zawierający wygenerowane dane, zostaje zapisany w katalogu *dane_testowe*, znajdującym się w tym samym katalogu co generator.

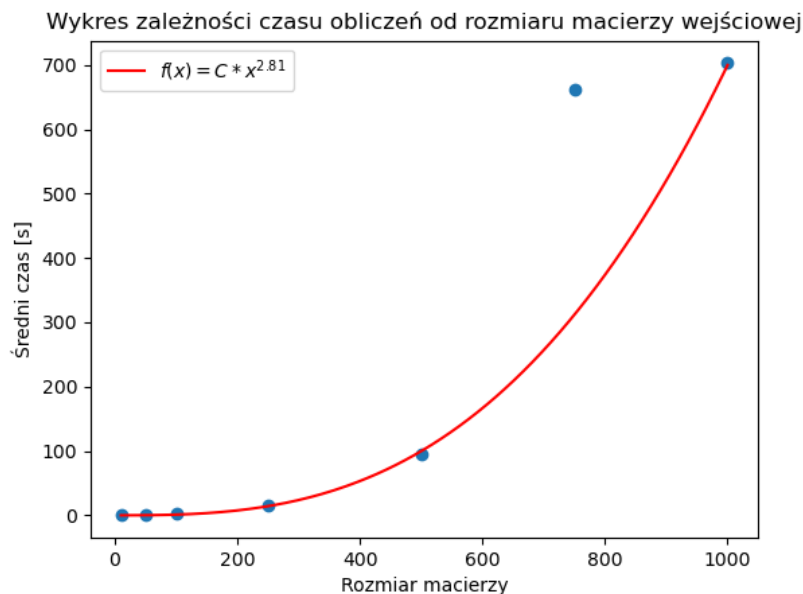
Uwaga: jeżeli katalog *dane_testowe* nie zostanie wykryty, program zwróci błąd.

3 Opis programu

Program działa w następujący sposób. Na samym początku użytkownik zostaje zapytany, czy chce załadować macierz zwycięstw z pliku, czy wprowadzić wyniki meczów ręcznie. Po wybraniu opcji „Dane z pliku” użytkownik ma opcje wybrania pliku z folderu, w którym znajduje się aplikacja, lub z podfolderu *dane_testowe*, w którym znajduje się 70 różnych danych wejściowych, które były używane do testowania aplikacji. Po wybraniu pliku aplikacja wyświetla macierz zwycięstw i listę zawodników z własnością X. Przy wybraniu opcji „Dane wpisane ręcznie” użytkownik najpierw jest proszony o podanie liczby zawodników, a następnie o wyniki każdego z meczów. Następnie aplikacja wyświetla na ich podstawie macierz zwycięstw i wypisuje zawodników o własności X. Na koniec użytkownik zostaje zapytany, czy chce zapisać wynik do pliku, a po wybraniu opcji „tak”, program zapisuje plik o podanej przez użytkownika nazwie w folderze *wyniki*.

4 Testy

Program został przetestowany przy użyciu wbudowanego modułu `time` w języku Python. Wykonano po 10 pomiarów dla każdej z następujących liczb zawodników: 10, 50, 100, 250, 500, 750, 1000. Średnie wartości czasu potrzebnego do obliczeń dla danego rozmiaru macierzy wejściowej zostały naniesione na wykres. Do punktów dopasowano krzywą $f(x) = Cx^{2.81}$ (gdzie $C = 2,6 \cdot 10^{-6}$ [s]), dzięki której mamy możliwość porównania złożoności naszego algorytmu do teoretycznej złożoności algorytmu Strassena.



Nieco niepokojąco może na wykresie wyglądać punkt (750, 662.7[s]). Taki wynik bierze się z charakteru algorytmu Strassena, który implementuje się dla macierzy, których rozmiary są potęgami liczby 2. Pozostałe macierze dopełnia się zerami do macierzy rozmiaru $m \times m$, gdzie m jest potęgą dwójki. Najbliższą potęgą dwójki większą od 750 jest $1024 = 2^{10}$, co sprawia, że zamiast macierzy 750×750 mnożymy macierze 1024×1024 , których 274 prawe kolumny i 274 dolne wiersze są pełne zer. Stąd zbliżony czas do macierzy rozmiaru 1000, dla których najbliższą potęgą liczby 2 jest również 1024. Wszystkie pozostałe rozmiary macierzy, które mają swoje punkty na wykresie, poza macierzą rozmiaru 750, są znacznie bliżej potęgi liczby 2, dlatego lepiej pasują do teoretycznej złożoności algorytmu Strassena.

5 Wnioski z testów

Na podstawie przeprowadzonych testów można zauważyć, że czas działania algorytmu rośnie w sposób nieliniowy wraz ze wzrostem liczby zawodników n . W szczególności dla większych wartości n obserwowany jest bardzo szybki wzrost czasu obliczeń, co jest zgodne z oczekiwaniami wynikającymi z analizy teoretycznej.

Otrzymane wyniki eksperymentalne są spójne z przewidywaną złożonością obliczeniową algorytmu Strassena wynoszącą $O(n^{\log_2 7})$. Krzywa wartości eksperymentalnych wykazuje podobny trend wzrostu jak funkcja teoretyczna, co potwierdza poprawność implementacji algorytmu.

6 Podział prac

- Ewelina Preś
 - pomysł zastosowania algorytmu szybkiego mnożenia macierzy
 - implementacja algorytmu Strassena i znajdowania zawodników o własności X na podstawie zadanej macierzy
 - przeprowadzenie testów
 - praca nad dokumentacją wstępną i końcową
- Stanisław Boczarski
 - pseudokod algorytmów
 - implementacja wersji beta aplikacji (wczytywanie macierzy podanej przez użytkownika i zwracanie rozwiązania)
 - przeprowadzenie testów
 - praca nad dokumentacją wstępną i końcową
- Dominik Dygas
 - pomysł usprawnienia algorytmu (jedynki na przekątnej zamiast zer)
 - wstępna implementacja rozwiązania problemu (nie została użyta - znaleźliśmy usprawnienie i oddaliśmy inną wersję implementacji)
 - generator plików wejściowych
 - praca nad dokumentacją wstępną i końcową